```
import { useState, useEffect, useRef, useCallback } from "react";

/* ═══════════════════════════════════════════════════════════════
   SSFFMVP v3 — Slops Saloon Fantasy Football MVP
   - Rebranded: Slops Saloon Fantasy Football MVP
   - DB Security: token encryption via Supabase Vault (documented)
   - Redis caching layer for standings + scores (documented)
   - Human-in-the-Loop "Did You Follow This?" confirmation toggle
   - Cron calibration only scores moves user actually executed
═══════════════════════════════════════════════════════════════ */

const CSS = `
@import url('https://fonts.googleapis.com/css2?family=Barlow+Condensed:wght@400;6

:root {
  --bg:      #07090E;
  --surface: #0E1319;
  --panel:   #131B24;
  --border:  #1C2B38;
  --border2: #243344;
  --accent:  #00D68F;
  --gold:    #F5C542;
  --red:     #FF4D6A;
  --blue:    #3D9BFF;
  --text:    #B8CDD8;
  --bright:  #E8F4FC;
  --muted:   #3D5468;
  --font-d:  'Barlow Condensed', sans-serif;
  --font-b:  'Barlow', sans-serif;
}

*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
html { scroll-behavior: smooth; }
body { background: var(--bg); font-family: var(--font-b); color: var(--text); min

/* ── GRID BG ── */
.app { min-height: 100vh; position: relative; }
.app::before {
  content: '';
  position: fixed; inset: 0;
  background-image:
    linear-gradient(rgba(0,214,143,.025) 1px, transparent 1px),
    linear-gradient(90deg, rgba(0,214,143,.025) 1px, transparent 1px);
  background-size: 48px 48px;
```

```
  pointer-events: none; z-index: 0;
}

/* ── TYPOGRAPHY ── */
.display { font-family: var(--font-d); letter-spacing: 1px; }

/* ── SHARED ── */
.tag {
  display: inline-flex; align-items: center; gap: 6px;
  font-size: 10px; letter-spacing: 3px; text-transform: uppercase;
  padding: 4px 12px; border-radius: 3px;
}
.tag-green { background: rgba(0,214,143,.1); border: 1px solid rgba(0,214,143,.25
.tag-gold  { background: rgba(245,197,66,.1); border: 1px solid rgba(245,197,66,.
.tag-red   { background: rgba(255,77,106,.1); border: 1px solid rgba(255,77,106,.
.tag-blue  { background: rgba(61,155,255,.1); border: 1px solid rgba(61,155,255,.

.btn {
  font-family: var(--font-b); font-weight: 600; font-size: 13px;
  letter-spacing: 1px; text-transform: uppercase;
  padding: 13px 30px; border-radius: 6px; border: none;
  cursor: pointer; transition: all .2s; outline: none;
}
.btn-primary { background: var(--accent); color: var(--bg); }
.btn-primary:hover:not(:disabled) { background: #00f0a0; transform: translateY(-2
.btn-primary:disabled { opacity: .4; cursor: not-allowed; transform: none; }
.btn-ghost { background: transparent; color: var(--text); border: 1px solid var(-
.btn-ghost:hover { border-color: var(--accent); color: var(--accent); }
.btn-danger { background: transparent; color: var(--muted); border: 1px solid var
.btn-danger:hover { color: var(--red); border-color: var(--red); }

/* ── LANDING ── */
.land {
  position: relative; z-index: 1;
  min-height: 100vh;
  display: flex; flex-direction: column; align-items: center; justify-content: ce
  padding: 80px 24px 60px;
  text-align: center;
}
.orb-wrap { position: fixed; top: -120px; left: 50%; transform: translateX(-50%);
.orb {
  width: 700px; height: 700px; border-radius: 50%;
  background: radial-gradient(circle, rgba(0,214,143,.07) 0%, transparent 65%);
  animation: breathe 6s ease-in-out infinite;
}
@keyframes breathe { 0%,100%{transform:scale(1);opacity:.7} 50%{transform:scale(1
```

```
.land-eyebrow { margin-bottom: 28px; }
.land-logo {
  font-family: var(--font-d); font-weight: 700;
  font-size: clamp(88px, 19vw, 164px);
  letter-spacing: 8px; line-height: .95;
  background: linear-gradient(128deg, #fff 0%, var(--accent) 50%, var(--gold) 100
  -webkit-background-clip: text; -webkit-text-fill-color: transparent; background
  margin-bottom: 6px;
}
.land-name {
  font-size: 11px; letter-spacing: 5px; text-transform: uppercase; color: var(--m
  margin-bottom: 40px;
}
.land-h {
  font-size: clamp(22px, 4vw, 32px); font-weight: 300; color: var(--bright);
  max-width: 560px; line-height: 1.45; margin-bottom: 14px;
}
.land-h strong { font-weight: 600; color: var(--accent); }
.land-p { font-size: 14px; color: var(--muted); max-width: 420px; line-height: 1.

.land-btns { display: flex; gap: 14px; justify-content: center; flex-wrap: wrap;

.feat-strip {
  display: flex; gap: 1px; background: var(--border);
  border: 1px solid var(--border); border-radius: 10px; overflow: hidden;
  max-width: 700px; width: 100%;
}
.feat-cell { background: var(--panel); flex: 1; padding: 26px 20px; text-align: c
.feat-ico { font-size: 24px; margin-bottom: 10px; }
.feat-lbl { font-size: 10px; letter-spacing: 2px; text-transform: uppercase; colo
.feat-val { font-size: 14px; font-weight: 600; color: var(--bright); }

/* ── ONBOARDING ── */
.ob { position: relative; z-index: 1; min-height: 100vh; display: flex; align-ite
.ob-card { background: var(--panel); border: 1px solid var(--border2); border-rad
.ob-step { margin-bottom: 8px; }
.ob-h { font-family: var(--font-d); font-size: 44px; font-weight: 700; color: var
.ob-p { font-size: 13px; color: var(--muted); line-height: 1.65; margin-bottom: 3

.plat-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; margin-bot
.plat-card {
  background: var(--bg); border: 1px solid var(--border); border-radius: 10px;
  padding: 20px 16px; cursor: pointer; transition: all .2s; text-align: center;
}
.plat-card:hover { border-color: var(--border2); background: var(--surface); }
.plat-card.active { border-color: var(--accent); background: rgba(0,214,143,.04);
.plat-ico { font-size: 28px; margin-bottom: 8px; }
```

```
.plat-name { font-size: 14px; font-weight: 600; color: var(--bright); }
.plat-note { font-size: 10px; color: var(--muted); margin-top: 3px; }

.form-stack { display: flex; flex-direction: column; gap: 16px; margin-bottom: 28
.form-field { display: flex; flex-direction: column; gap: 7px; }
.form-label { font-size: 10px; letter-spacing: 2.5px; text-transform: uppercase;
.form-input, .form-select {
  background: var(--bg); border: 1px solid var(--border); border-radius: 7px;
  padding: 12px 16px; font-family: var(--font-b); font-size: 14px; color: var(--b
  outline: none; transition: border-color .2s; width: 100%;
}
.form-input:focus, .form-select:focus { border-color: var(--accent); }
.form-input.error { border-color: var(--red); }
.form-select option { background: var(--bg); }
.form-err { font-size: 11px; color: var(--red); margin-top: -8px; }
.ob-btns { display: flex; gap: 10px; }

/* ── TOP BAR ── */
.topbar {
  position: sticky; top: 0; z-index: 200;
  display: flex; align-items: center; justify-content: space-between;
  padding: 14px 28px;
  background: rgba(7,9,14,.93); backdrop-filter: blur(16px);
  border-bottom: 1px solid var(--border);
}
.tb-logo {
  font-family: var(--font-d); font-size: 24px; font-weight: 700; letter-spacing:
  background: linear-gradient(120deg, var(--bright), var(--accent));
  -webkit-background-clip: text; -webkit-text-fill-color: transparent; background
}
.tb-right { display: flex; align-items: center; gap: 14px; }
.tb-week { font-size: 10px; letter-spacing: 2px; text-transform: uppercase; }
.tb-team { font-size: 13px; color: var(--muted); max-width: 140px; overflow: hidd
.tb-rec { font-weight: 700; font-size: 13px; color: var(--gold); }
.sub-pill {
  background: linear-gradient(120deg, var(--accent), #00a878);
  color: var(--bg); font-size: 10px; font-weight: 700; letter-spacing: 1.5px;
  text-transform: uppercase; padding: 5px 12px; border-radius: 4px; border: none;
  cursor: pointer; transition: all .2s;
}
.sub-pill:hover { opacity: .85; transform: translateY(-1px); }
.sub-pill.locked { background: var(--surface); color: var(--muted); border: 1px s

/* ── DASHBOARD BODY ── */
.dash { position: relative; z-index: 1; }
.dash-grid {
  display: grid;
```

```
  grid-template-columns: 1fr 350px;
  gap: 22px;
  max-width: 1140px; margin: 0 auto;
  padding: 28px 24px;
}
.sec-head { margin-bottom: 14px; }
.sec-lbl { font-size: 10px; letter-spacing: 3px; text-transform: uppercase; color

/* ── SUBSCRIPTION GATE ── */
.gate {
  background: var(--panel); border: 1px solid var(--border2); border-radius: 14px
  padding: 48px 36px; text-align: center; margin-bottom: 22px;
  position: relative; overflow: hidden;
}
.gate::before {
  content: '';
  position: absolute; inset: 0;
  background: repeating-linear-gradient(45deg, transparent, transparent 20px, rgb
}
.gate-lock { font-size: 44px; margin-bottom: 16px; }
.gate-h { font-family: var(--font-d); font-size: 36px; font-weight: 700; letter-s
.gate-p { font-size: 14px; color: var(--muted); max-width: 320px; margin: 0 auto
.gate-features { display: flex; gap: 24px; justify-content: center; flex-wrap: wr
.gate-feat { display: flex; align-items: center; gap: 7px; font-size: 13px; color
.gate-feat::before { content: '✓'; color: var(--accent); font-weight: 700; }

/* ── AGENT PANEL ── */
.agent-panel {
  background: var(--panel); border: 1px solid var(--border2); border-radius: 14px
  overflow: hidden; margin-bottom: 22px;
  position: relative;
}
.agent-scan {
  position: absolute; top: 0; left: 0; right: 0; height: 2px;
  background: linear-gradient(90deg, transparent 0%, var(--accent) 50%, transpare
  animation: scan 2.5s linear infinite;
}
@keyframes scan { 0%{transform:translateX(-100%)} 100%{transform:translateX(100%)

.agent-header { padding: 22px 26px; border-bottom: 1px solid var(--border); displ
.agent-title { display: flex; align-items: center; gap: 10px; }
.pulse { width: 7px; height: 7px; border-radius: 50%; background: var(--accent);
@keyframes p { 0%,100%{opacity:1;transform:scale(1)} 50%{opacity:.25;transform:sc
.agent-ttl-txt { font-size: 11px; letter-spacing: 3px; text-transform: uppercase;
.agent-prog-txt { font-size: 12px; color: var(--muted); }

.agent-steps { padding: 20px 22px; display: flex; flex-direction: column; gap: 10
```

```
.a-step {
  display: flex; align-items: flex-start; gap: 14px;
  padding: 14px 16px; background: var(--bg);
  border: 1px solid var(--border); border-radius: 9px;
  transition: all .35s;
}
.a-step.s-done  { border-color: rgba(0,214,143,.3); }
.a-step.s-active{ border-color: var(--accent); background: rgba(0,214,143,.04); }
.a-step.s-error { border-color: rgba(255,77,106,.4); background: rgba(255,77,106,
.a-step-ico { font-size: 20px; flex-shrink: 0; margin-top: 1px; }
.a-step-body { flex: 1; }
.a-step-name { font-size: 13px; font-weight: 600; color: var(--bright); margin-bo
.a-step-src { font-size: 10px; letter-spacing: 1px; text-transform: uppercase; co
.a-step-status { font-size: 12px; color: var(--muted); line-height: 1.4; }
.a-step-status.s-done   { color: var(--accent); }
.a-step-status.s-active { color: var(--gold); }
.a-step-status.s-error  { color: var(--red); }

.verdict-box {
  margin: 0 22px 22px; padding: 20px;
  background: rgba(0,214,143,.06); border: 1px solid rgba(0,214,143,.2); border-r
  animation: fadeUp .5s ease;
}
.verdict-lbl { font-size: 10px; letter-spacing: 3px; text-transform: uppercase; c
.verdict-txt { font-size: 14px; color: var(--bright); line-height: 1.65; }
.verdict-meta { margin-top: 10px; font-size: 11px; color: var(--muted); display:
.verdict-meta span { display: flex; align-items: center; gap: 5px; }

/* ── MOVE CARD ── */
.move-card {
  background: var(--panel); border: 1px solid var(--border2); border-radius: 14px
  overflow: hidden; margin-bottom: 22px; animation: fadeUp .4s ease;
}
@keyframes fadeUp { from{opacity:0;transform:translateY(16px)} to{opacity:1;trans

.mc-head {
  padding: 24px 28px;
  background: linear-gradient(135deg, rgba(0,214,143,.09) 0%, rgba(245,197,66,.04
  border-bottom: 1px solid var(--border);
  display: flex; align-items: flex-start; justify-content: space-between; gap: 16
}
.mc-left { flex: 1; }
.mc-eyebrow { display: flex; gap: 8px; flex-wrap: wrap; margin-bottom: 12px; }
.mc-h { font-family: var(--font-d); font-size: 34px; font-weight: 700; letter-spa
.mc-scoring { font-size: 11px; color: var(--muted); margin-top: 6px; }
.mc-conf { text-align: right; flex-shrink: 0; }
.mc-conf-lbl { font-size: 10px; letter-spacing: 2px; text-transform: uppercase; c
```

```
.mc-conf-num { font-family: var(--font-d); font-size: 52px; font-weight: 700; lin
.mc-conf-of  { font-size: 11px; color: var(--muted); }

.mc-body { padding: 26px 28px; }
.mc-reasoning { font-size: 14px; color: var(--text); line-height: 1.75; margin-bo

.signals { display: grid; grid-template-columns: repeat(3,1fr); gap: 10px; margin
.sig {
  background: var(--bg); border: 1px solid var(--border); border-radius: 9px;
  padding: 16px 14px; transition: border-color .2s;
}
.sig:hover { border-color: var(--border2); }
.sig-ico  { font-size: 20px; margin-bottom: 8px; }
.sig-lbl  { font-size: 10px; letter-spacing: 1.5px; text-transform: uppercase; co
.sig-val  { font-size: 13px; font-weight: 600; color: var(--bright); margin-botto
.sig-sub  { font-size: 11px; color: var(--muted); line-height: 1.3; }

.mc-actions { display: flex; gap: 10px; align-items: center; flex-wrap: wrap; }
.mc-note { font-size: 11px; color: var(--muted); margin-left: auto; }

/* ── CONFIRMED / SKIPPED ── */
.state-card {
  background: var(--panel); border: 1px solid var(--border2); border-radius: 14px
  padding: 44px; text-align: center; margin-bottom: 22px; animation: fadeUp .4s e
}
.state-ico { font-size: 52px; margin-bottom: 16px; }
.state-h { font-family: var(--font-d); font-size: 32px; font-weight: 700; letter-
.state-p { font-size: 14px; color: var(--muted); line-height: 1.65; max-width: 36
.feedback-prompt { background: var(--bg); border: 1px solid var(--border); border
.fb-lbl { font-size: 11px; letter-spacing: 2px; text-transform: uppercase; color:
.fb-stars { display: flex; gap: 8px; margin-bottom: 14px; }
.fb-star { font-size: 24px; cursor: pointer; transition: transform .15s; filter:
.fb-star:hover, .fb-star.lit { filter: none; transform: scale(1.1); }
.fb-inp {
  width: 100%; background: var(--surface); border: 1px solid var(--border); borde
  padding: 10px 14px; font-family: var(--font-b); font-size: 13px; color: var(--b
  outline: none; resize: none; height: 72px; transition: border-color .2s;
}
.fb-inp:focus { border-color: var(--accent); }
.fb-submit { margin-top: 12px; }

/* ── HISTORY ── */
.hist-panel { background: var(--panel); border: 1px solid var(--border2); border-
.hist-head { padding: 16px 22px; border-bottom: 1px solid var(--border); display:
.hist-ttl { font-size: 10px; letter-spacing: 3px; text-transform: uppercase; colo
.hist-stat { font-size: 12px; color: var(--accent); }
.hist-list { max-height: 440px; overflow-y: auto; }
```

```
.hist-list::-webkit-scrollbar { width: 3px; }
.hist-list::-webkit-scrollbar-thumb { background: var(--border); border-radius: 2
.hist-item { padding: 16px 22px; border-bottom: 1px solid var(--border); transiti
.hist-item:last-child { border-bottom: none; }
.hist-item:hover { background: rgba(255,255,255,.015); }
.hi-top { display: flex; align-items: flex-start; justify-content: space-between;
.hi-wk { font-size: 10px; letter-spacing: 2px; text-transform: uppercase; color:
.hi-eff { font-size: 12px; color: var(--muted); flex-shrink: 0; }
.hi-mv { font-size: 13px; color: var(--bright); line-height: 1.4; margin-bottom:
.hi-out { font-size: 11px; color: var(--muted); margin-bottom: 6px; display: flex
.hi-win  { color: var(--accent); font-weight: 600; }
.hi-loss { color: var(--red);    font-weight: 600; }
.hi-pend { color: var(--gold);   font-weight: 600; }
.eff-track { height: 3px; background: var(--border); border-radius: 99px; overflo
.eff-fill  { height: 100%; border-radius: 99px; transition: width .6s ease; }

/* ── STANDS ── */
.stand-panel { background: var(--panel); border: 1px solid var(--border2); border
.stand-head { padding: 16px 22px; border-bottom: 1px solid var(--border); }
.stand-lbl { font-size: 10px; letter-spacing: 3px; text-transform: uppercase; col
.stand-row {
  display: grid; grid-template-columns: 28px 1fr 52px 60px;
  gap: 10px; padding: 12px 22px; border-bottom: 1px solid var(--border);
  align-items: center; font-size: 13px; transition: background .15s;
}
.stand-row:last-child { border-bottom: none; }
.stand-row:hover { background: rgba(255,255,255,.015); }
.stand-row.me { background: rgba(0,214,143,.04); border-left: 2px solid var(--acc
.s-rank { font-size: 11px; color: var(--muted); text-align: center; }
.s-name { color: var(--bright); font-weight: 500; overflow: hidden; text-overflow
.s-rec  { color: var(--muted); font-size: 12px; text-align: center; }
.s-pts  { color: var(--accent); font-size: 12px; text-align: right; font-weight:

/* ── SUB MODAL ── */
.overlay {
  position: fixed; inset: 0; z-index: 999;
  background: rgba(0,0,0,.85); backdrop-filter: blur(12px);
  display: flex; align-items: center; justify-content: center; padding: 20px;
  animation: fadeIn .2s ease;
}
@keyframes fadeIn { from{opacity:0} to{opacity:1} }
.modal {
  background: var(--panel); border: 1px solid var(--border2); border-radius: 18px
  padding: 52px 48px; width: 100%; max-width: 560px; position: relative;
  animation: slideUp .3s ease;
}
@keyframes slideUp { from{transform:translateY(24px);opacity:0} to{transform:tran
```

```
.modal-x {
  position: absolute; top: 18px; right: 18px;
  background: var(--bg); border: 1px solid var(--border); border-radius: 6px;
  width: 32px; height: 32px; display: flex; align-items: center; justify-content:
  color: var(--muted); font-size: 16px; cursor: pointer; transition: all .2s;
}
.modal-x:hover { color: var(--bright); border-color: var(--border2); }
.modal-eyebrow { margin-bottom: 10px; }
.modal-h { font-family: var(--font-d); font-size: 52px; font-weight: 700; letter-
.modal-p { font-size: 14px; color: var(--muted); line-height: 1.65; margin-bottom

.plan-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; margin-bot
.plan {
  background: var(--bg); border: 1px solid var(--border); border-radius: 12px;
  padding: 26px 22px; cursor: pointer; transition: all .2s; position: relative;
}
.plan:hover { border-color: var(--border2); }
.plan.best { border-color: var(--accent); box-shadow: 0 0 0 1px var(--accent); }
.plan-badge {
  position: absolute; top: -11px; left: 50%; transform: translateX(-50%);
  background: var(--accent); color: var(--bg);
  font-size: 9px; font-weight: 700; letter-spacing: 2px; text-transform: uppercas
  padding: 3px 12px; border-radius: 99px;
}
.plan-name { font-size: 10px; letter-spacing: 2px; text-transform: uppercase; col
.plan-price { font-family: var(--font-d); font-size: 44px; font-weight: 700; colo
.plan-per { font-size: 12px; color: var(--muted); margin-bottom: 18px; }
.plan-feats { list-style: none; display: flex; flex-direction: column; gap: 8px;
.plan-feat { font-size: 12px; color: var(--text); display: flex; align-items: cen
.plan-feat::before { content: '✓'; color: var(--accent); font-weight: 700; flex-s
.modal-fine { font-size: 11px; color: var(--muted); text-align: center; margin-to

/* ── ARCH DIAGRAM ── */
.arch-banner {
  background: var(--panel); border: 1px solid var(--border2); border-radius: 14px
  padding: 24px; margin-bottom: 22px;
}
.arch-ttl { font-size: 10px; letter-spacing: 3px; text-transform: uppercase; colo
.arch-flow { display: flex; align-items: center; gap: 0; flex-wrap: wrap; }
.arch-node {
  background: var(--bg); border: 1px solid var(--border); border-radius: 8px;
  padding: 10px 14px; font-size: 12px; color: var(--text); white-space: nowrap;
  display: flex; align-items: center; gap: 6px;
}
.arch-node.highlight { border-color: var(--accent); color: var(--accent); }
.arch-arrow { color: var(--muted); font-size: 14px; padding: 0 6px; flex-shrink:
```

```
/* ── TOAST ── */
.toast {
  position: fixed; bottom: 28px; left: 50%;
  transform: translateX(-50%) translateY(80px);
  background: var(--panel); border: 1px solid var(--border2);
  border-radius: 9px; padding: 13px 24px;
  font-size: 13px; color: var(--bright);
  z-index: 9999; transition: transform .4s cubic-bezier(.34,1.56,.64,1);
  white-space: nowrap; display: flex; align-items: center; gap: 10px;
  box-shadow: 0 8px 32px rgba(0,0,0,.4);
}
.toast.show { transform: translateX(-50%) translateY(0); }
.toast.t-green { border-color: rgba(0,214,143,.4); }
.toast.t-red   { border-color: rgba(255,77,106,.4); }

/* ── UTIL ── */
.divider { height: 1px; background: var(--border); margin: 0 0 22px; }

@media (max-width: 820px) {
  .dash-grid { grid-template-columns: 1fr; }
  .signals { grid-template-columns: 1fr 1fr; }
  .plan-grid { grid-template-columns: 1fr; }
  .feat-strip { flex-direction: column; }
  .mc-head { flex-direction: column; }
  .ob-card { padding: 32px 24px; }
  .modal { padding: 36px 24px; }
  .arch-flow { gap: 4px; }
}
`;

/* ═══════════════════════════════════════════════════════════════
   PLATFORM CONFIG
═══════════════════════════════════════════════════════════════ */
const PLATFORMS = [
  { id:"sleeper", ico:"💤", name:"Sleeper",  note:"Username-based OAuth" },
  { id:"yahoo",   ico:"🟣", name:"Yahoo!",   note:"OAuth2 required" },
  { id:"espn",    ico:"🏈", name:"ESPN",     note:"Backend scraper" },
  { id:"nfl",     ico:"🏟️", name:"NFL.com",  note:"League ID required" },
];

const AGENT_DEFS = [
  { id:"weather",   ico:"🌦️", name:"Weather Agent",     src:"OpenWeatherMap API",
  { id:"travel",    ico:"✈️", name:"Travel Agent",       src:"Schedule + Maps API
  { id:"gametime",  ico:"🕐", name:"Game Time Agent",    src:"NFL Schedule API",
  { id:"roster",    ico:"📋", name:"Roster Agent",       src:"Sportradar / Tank01
  { id:"perf",      ico:"📈", name:"Performance Agent",  src:"Sportradar Stats",
  { id:"matchup",   ico:"⚔️", name:"Matchup Agent",      src:"DvP Rankings",
```

```
];

/* ═══════════════════════════════════════════════════════════════
   DATA LAYER — v3
   ─────────────────────────────────────────────────────────────

   PRODUCTION REPLACEMENT MAP:

   ┌──────────────────────┬──────────────────────────────────────────────┐
   │ fetchHistory()       │ GET /api/moves?userId=X                      │
   │                      │ → Supabase: SELECT * FROM moves WHERE user_id=X      │
   │                      │   ORDER BY week_num DESC                     │
   ├──────────────────────┼──────────────────────────────────────────────┤
   │ fetchStandings()     │ Sleeper:  GET /league/{league_id}/rosters    │
   │                      │ Yahoo:    GET /fantasy/v2/league/{key}/standings     │
   │                      │ ESPN:     GET /apis/v3/games/ffl/seasons/{yr}/       │
   │                      │           segments/0/leagues/{id}?view=mStandings   │
   ├──────────────────────┼──────────────────────────────────────────────┤
   │ Outcome Scoring      │ Tuesday cron (Railway/Render):               │
   │                      │   1. Only score moves where followed=true    │
   │                      │   2. Fetch prev week scores via Sportradar   │
   │                      │   3. Calculate eff = composite score 0-100   │
   │                      │   4. PATCH /api/moves/:id { outcome, eff }   │
   │                      │   5. Feed eff history into next week's prompt        │
   └──────────────────────┴──────────────────────────────────────────────┘

   ── SECURITY (v3) ──────────────────────────────────────────────
   OAuth access_tokens and ESPN cookies must NEVER be stored as
   plaintext in Supabase. Use Supabase Vault (pgsodium):

     -- Enable Vault extension once:
     CREATE EXTENSION IF NOT EXISTS pgsodium;

     -- Encrypt on write (in your backend route):
     UPDATE platform_connections
     SET access_token = pgsodium.crypto_secretbox(
       convert_to(raw_token, 'utf8'),
       pgsodium.crypto_secretbox_keygen()  -- store key in Railway env, not DB
     )
     WHERE user_id = $1;

     -- Decrypt on read:
     SELECT convert_from(
       pgsodium.crypto_secretbox_open(access_token, $key),
       'utf8'
     ) AS raw_token FROM platform_connections WHERE user_id = $1;

   Alternative: Use Supabase Vault's built-in secrets manager
   (Dashboard → Vault → New Secret) and reference by name only.
```

```
   ── REDIS CACHING LAYER (v3) ──────────────────────────────────
   Problem: 100 users in the same Sleeper league hitting the API
   simultaneously = 100 identical calls + Sportradar rate-limit hits.

   Solution: Redis cache keyed by league_id, TTL = 5 min for standings,
   60 min for NFL scores (scores don't change mid-week).

   // In your Express standings route (/api/league/standings):
   const redis   = new Redis(process.env.REDIS_URL);
   const cacheKey = `standings:${platform}:${leagueId}`;
   const cached   = await redis.get(cacheKey);
   if (cached) return res.json(JSON.parse(cached));       // cache HIT

   const live = await buildStandings(platform, leagueId); // cache MISS
   await redis.setex(cacheKey, 300, JSON.stringify(live)); // 5-min TTL
   return res.json(live);

   // In Tuesday cron — Sportradar scores (cache 60 min):
   const scoreKey = `nfl:scores:${season}:week${weekNum}`;
   const cached   = await redis.get(scoreKey);
   const scores   = cached ? JSON.parse(cached) : await fetchNFLScores(weekNum);
   if (!cached) await redis.setex(scoreKey, 3600, JSON.stringify(scores));

   Deploy Redis: Upstash (serverless Redis, free tier, Railway add-on)
   or Railway Redis plugin. Connection string goes in REDIS_URL env var.
═══════════════════════════════════════════════════════════════ */

// ── FALLBACK SEEDS (shown while real data loads or if API fails) ──
const HISTORY_FALLBACK = [
  { id:1, week:"Week 14", wkNum:14, move:"Start D. Henry over A. Jones — power ba
  { id:2, week:"Week 13", wkNum:13, move:"Pickup: Jaylen Warren — beat waiver wir
  { id:3, week:"Week 12", wkNum:12, move:"Bench CeeDee Lamb — elite CB matchup +
  { id:4, week:"Week 11", wkNum:11, move:"Trade: Chase + pick → Kelce + depth pie
  { id:5, week:"Week 10", wkNum:10, move:"Stack Mahomes + Kelce as locked core",
];

const STANDINGS_FALLBACK = [
  { rank:1, name:"Gridiron Gods",     rec:"11-3", pts:"1842", me:false },
  { rank:2, name:"PLACEHOLDER",       rec:"10-4", pts:"1791", me:true  },
  { rank:3, name:"Touchdown Factory", rec:"9-5",  pts:"1730", me:false },
  { rank:4, name:"Blitz Brigade",     rec:"8-6",  pts:"1698", me:false },
  { rank:5, name:"Red Zone Rangers",  rec:"7-7",  pts:"1645", me:false },
];

// ── DATA FETCH FUNCTIONS ──
// Each function shows exactly what the production call looks like.
```

```
// Swap the `return simulateLatency(FALLBACK)` line for the real fetch().

async function simulateLatency(data, ms = 800) {
  await new Promise(r => setTimeout(r, ms));
  return data;
}

async function fetchHistory(userId, platform) {
  // PRODUCTION:
  // const r = await fetch(`/api/moves?userId=${userId}`);
  // const d = await r.json();
  // return d.moves;   // [{id, week, wkNum, move, type, outcome, result, eff, sc
  return simulateLatency(HISTORY_FALLBACK, 600);
}

async function fetchStandings(platform, leagueId, teamName) {
  // PRODUCTION — all routed through your backend which checks Redis first:
  // const r = await fetch(`/api/league/standings?platform=${platform}&leagueId=$
  // return r.json();  // Redis cache hit = instant; miss = live API call + cache
  //
  // Sleeper direct (no auth — still benefits from Redis de-duplication):
  // const rosters = await fetch(`https://api.sleeper.app/v1/league/${leagueId}/r
  // const users   = await fetch(`https://api.sleeper.app/v1/league/${leagueId}/u
  // return buildStandingsFromRosters(rosters, users, teamName);
  //
  // Yahoo (requires OAuth bearer token from backend):
  // const r = await fetch(`/api/yahoo/standings?leagueId=${leagueId}`);
  // return r.json();
  //
  // ESPN (backend cookie scraper):
  // const r = await fetch(`/api/espn/standings?leagueId=${leagueId}`);
  // return r.json();
  const rows = STANDINGS_FALLBACK.map(r => r.me ? {...r, name: teamName || "Your
  return simulateLatency(rows, 700);
}

/* ═══════════════════════════════════════
   SECURE API BRIDGE
   v2: In production this hits YOUR backend (/api/move)
   which holds the Anthropic key server-side.
   Here we call the API directly for prototype purposes,
   but the architecture is designed for the proxy pattern.
═══════════════════════════════════════ */
async function callManagerAgent(context) {
  const sys = `You are the Slops Saloon Fantasy Football MVP Manager Agent — an e
You receive structured intelligence from 6 sub-agents and synthesize ONE perfect
Rules:
```

```
- Output ONLY valid JSON. No markdown, no preamble.
- Format: { "moveType": string, "headline": string, "confidence": number(0-100),
- moveType: one of ["Waiver Pickup","Start/Sit","Trade","Stack","Drop","Lineup Tw
- headline: 8 words max, action-oriented
- confidence: integer, honest (not always high)
- reasoning: 2-3 sentences, cite the agent signals, mention scoring format
- shortVerdict: 1 sentence sharp summary
Scoring format affects recommendations significantly — PPR favors pass-catchers.`

  const prompt = `Agent Intelligence Report:
WEATHER: ${context.weather}
TRAVEL: ${context.travel}
GAME TIME: ${context.gametime}
ROSTER: ${context.roster}
PERFORMANCE: ${context.perf}
MATCHUP: ${context.matchup}
SCORING FORMAT: ${context.scoring}
TEAM RECORD: ${context.record}
Generate the single best move this GM should make this week.`;

  try {
    const r = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        model: "claude-3-5-sonnet-latest", // production model — fast, cost-effic
        max_tokens: 1000,
        system: sys,
        messages: [{ role:"user", content: prompt }]
      })
    });
    const d = await r.json();
    const txt = d.content?.map(b => b.text||"").join("") || "";
    const clean = txt.replace(/```json|```/g,"").trim();
    return JSON.parse(clean);
  } catch {
    return {
      moveType: "Waiver Pickup",
      headline: "Add Jaylen Warren, Drop RB3",
      confidence: 87,
      reasoning: `Warren is absorbing 15+ carries weekly while Najee Harris manag
      shortVerdict: "Warren faces the worst run defense in the league — start him
    };
  }
}

/* ═══════════════════════════════════════════
```

```
   HOOKS
═══════════════════════════════════════════ */
function useToast() {
  const [toast, setToast] = useState({ show:false, msg:"", type:"t-green" });
  const show = (msg, type="t-green") => {
    setToast({ show:true, msg, type });
    setTimeout(() => setToast(t => ({...t, show:false})), 3200);
  };
  return [toast, show];
}

function usePersistentHistory(userId, platform) {
  const [history, setHistory] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  // On mount: try sessionStorage cache first, then fetch
  useEffect(() => {
    if (!userId) return;
    const cacheKey = `ssfmvp_history_${userId}`;
    let cached = null;
    try { cached = JSON.parse(sessionStorage.getItem(cacheKey)); } catch {}
    if (cached) { setHistory(cached); setLoading(false); }

    // Always re-fetch in background to stay fresh
    // PRODUCTION: fetchHistory calls GET /api/moves?userId=X (your backend → Sup
    fetchHistory(userId, platform)
      .then(rows => {
        setHistory(rows);
        try { sessionStorage.setItem(cacheKey, JSON.stringify(rows)); } catch {}
      })
      .catch(e => setError(e.message))
      .finally(() => setLoading(false));
  }, [userId, platform]);

  const save = (h) => {
    setHistory(h);
    try { sessionStorage.setItem(`ssfmvp_history_${userId}`, JSON.stringify(h));
    // PRODUCTION: POST /api/moves or PATCH /api/moves/:id
  };

  const addMove    = (move) => save([move, ...history]);
  const updateOutcome = (id, outcome, eff) => save(history.map(m => m.id===id ? {

  return [history, loading, error, addMove, updateOutcome];
}
```

```
function useStandings(platform, leagueId, teamName) {
  const [standings, setStandings] = useState([]);
  const [loading, setLoading]     = useState(true);
  const [error, setError]         = useState(null);

  useEffect(() => {
    if (!leagueId) return;
    // PRODUCTION: fetchStandings calls the real platform API via your backend
    fetchStandings(platform, leagueId, teamName)
      .then(rows => setStandings(rows))
      .catch(e => setError(e.message))
      .finally(() => setLoading(false));
  }, [platform, leagueId, teamName]);

  return [standings, loading, error];
}

/* ═══════════════════════════════════════
   COMPONENTS
═══════════════════════════════════════ */

function Toast({ toast }) {
  return (
    <div className={`toast ${toast.type} ${toast.show?"show":""}`}>
      {toast.type==="t-green" ? "✓" : "⚠"} {toast.msg}
    </div>
  );
}

// ── LANDING ──
function Landing({ onStart, onSub }) {
  return (
    <div className="land">
      <div className="orb-wrap"><div className="orb"/></div>
      <div className="land-eyebrow">
        <span className="tag tag-gold">🍺 Slops Saloon Presents</span>
      </div>
      <div className="land-logo">SSFFMVP</div>
      <div className="land-name">Slops Saloon Fantasy Football MVP</div>
      <p className="land-h">One <strong>perfect move</strong> every week.<br/>Bui
      <p className="land-p">Six specialized AI agents analyze weather, travel, ga
      <div className="land-btns">
        <button className="btn btn-primary" onClick={onStart}>Connect My League →
        <button className="btn btn-ghost" onClick={onSub}>View Plans</button>
      </div>
      <div className="feat-strip">
        <div className="feat-cell"><div className="feat-ico">🤖</div><div classNa
```

```
        <div className="feat-cell"><div className="feat-ico">🎯</div><div classNa
        <div className="feat-cell"><div className="feat-ico">🔄</div><div classNa
        <div className="feat-cell"><div className="feat-ico">🔒</div><div classNa
      </div>
    </div>
  );
}

// ── ONBOARDING v3 — Real OAuth Connect Buttons ──
//
// Three distinct flows depending on platform:
//   Sleeper → POST /api/auth/sleeper/connect (username lookup, no OAuth)
//   Yahoo   → GET  /api/auth/yahoo/authorize  (full OAuth 2.0 + PKCE redirect)
//   ESPN    → POST /api/auth/espn/connect     (ESPN_S2 + SWID cookie entry)
//
// After connection the backend returns leagues[] for the user to pick from.
// Scoring format auto-detected from platform where possible.
//
function Onboarding({ onDone }) {
  const [step,       setStep]       = useState(1);         // 1=platform, 2=conne
  const [plat,       setPlat]       = useState("");
  const [connStatus, setConnStatus] = useState("idle");    // idle | connecting |
  const [connError,  setConnError]  = useState("");
  const [leagues,    setLeagues]    = useState([]);
  const [chosenLeague, setChosen]   = useState(null);
  const [scoring,    setScoring]    = useState("PPR");

  // Platform-specific connection state
  const [sleeperUser, setSleeperUser] = useState("");
  const [espnS2,      setEspnS2]      = useState("");
  const [espnSWID,    setEspnSWID]    = useState("");
  const [espnLeagueId,setEspnLid]     = useState("");

  // ── Sleeper: username lookup ──────────────────────────
  const connectSleeper = async () => {
    if (!sleeperUser.trim()) return;
    setConnStatus("connecting");
    setConnError("");
    try {
      // PRODUCTION: POST /api/auth/sleeper/connect
      // const r = await fetch("/api/auth/sleeper/connect", {
      //   method:"POST", headers:{"Content-Type":"application/json"},
      //   body: JSON.stringify({ username: sleeperUser, userId: currentAppUserId
      // });
      // const d = await r.json();
      // if (!r.ok) throw new Error(d.error);
      // setLeagues(d.leagues);
```

```
    // PROTOTYPE SIMULATION:
    await new Promise(r => setTimeout(r, 1200));
    const mockLeagues = [
      { id:"1048209827491627008", name:"The Gridiron Gauntlet", size:10, scorin
      { id:"850283626509332480",  name:"Dynasty Destroyers",    size:12, scorin
    ];
    setLeagues(mockLeagues);
    setConnStatus("connected");
    setStep(3);
  } catch (e) {
    setConnStatus("error");
    setConnError(e.message || "Could not find Sleeper user. Check your username
  }
};

// ── Yahoo: OAuth redirect ──────────────────────────────
const connectYahoo = () => {
  setConnStatus("connecting");
  // PRODUCTION: window.location.href = `/api/auth/yahoo/authorize?userId=${cur
  // On return from OAuth callback, URL will contain ?connected=yahoo
  // and the backend will have saved the tokens.
  // For prototype, we simulate the return:
  setTimeout(() => {
    setLeagues([
      { id:"449.l.1283991", name:"Monday Night Mayhem", size:10, scoring:"PPR"
      { id:"449.l.9982774", name:"Yahoo Ballers",       size:12, scoring:"Stand
    ]);
    setConnStatus("connected");
    setStep(3);
  }, 1500);
};

// ── ESPN: cookie-based ─────────────────────────────────
const connectESPN = async () => {
  if (!espnS2 || !espnSWID || !espnLeagueId) {
    setConnError("All three ESPN fields are required.");
    return;
  }
  setConnStatus("connecting");
  setConnError("");
  try {
    // PRODUCTION:
    // const r = await fetch("/api/auth/espn/connect", {
    //   method:"POST", headers:{"Content-Type":"application/json"},
    //   body: JSON.stringify({ espnS2, swid:espnSWID, leagueId:espnLeagueId, u
    // });
```

```
    // const d = await r.json();
    // if (!r.ok) throw new Error(d.error);
    await new Promise(r => setTimeout(r, 1200));
    setLeagues([{ id: espnLeagueId, name:"My ESPN League", size:10, scoring:"St
    setConnStatus("connected");
    setStep(3);
  } catch (e) {
    setConnStatus("error");
    setConnError(e.message || "ESPN connection failed. Check your cookies and l
  }
};

const handleLeaguePick = (league) => {
  setChosen(league);
  // Auto-detect scoring from platform data
  if (league.scoring) setScoring(league.scoring);
  setStep(4);
};

const handleFinish = () => {
  if (!chosenLeague) return;
  onDone({
    platform:  plat,
    leagueId:  chosenLeague.id,
    leagueName:chosenLeague.name,
    teamName:  sleeperUser || "My Team",
    scoring,
  });
};

return (
  <div className="ob">
    <div className="orb-wrap"><div className="orb"/></div>

    {/* ── STEP 1: Platform select ── */}
    {step === 1 && (
      <div className="ob-card">
        <div className="ob-step"><span className="tag tag-green">Step 1 of 4</s
        <div className="ob-h">Choose Platform</div>
        <div className="ob-p">Each platform connects differently. Sleeper is in
        <div className="plat-grid">
          {PLATFORMS.map(p => (
            <div key={p.id} className={`plat-card${plat===p.id?" active":""}`}
              <div className="plat-ico">{p.ico}</div>
              <div className="plat-name">{p.name}</div>
              <div className="plat-note">{p.note}</div>
            </div>
```

```
      ))}
    </div>
    <button className="btn btn-primary" style={{width:"100%"}} disabled={!p
      Continue →
    </button>
  </div>
)}

{/* ── STEP 2: Platform-specific connect UI ── */}
{step === 2 && plat === "sleeper" && (
  <div className="ob-card">
    <div className="ob-step"><span className="tag tag-green">Step 2 of 4 —
    <div className="ob-h">Enter Username</div>
    <div className="ob-p">Sleeper's API is public — no login redirect neede
    <div className="form-stack">
      <div className="form-field">
        <label className="form-label">Sleeper Username</label>
        <input className={`form-input${connStatus==="error"?" error":""}`}
          value={sleeperUser} onChange={e=>setSleeperUser(e.target.value)}
          placeholder="your_sleeper_username" onKeyDown={e=>e.key==="Enter"
        {connStatus==="error" && <div className="form-err">⚠ {connError}</d
      </div>
    </div>
    <div className="ob-btns">
      <button className="btn btn-ghost" onClick={()=>setStep(1)}>← Back</bu
      <button className="btn btn-primary" style={{flex:1}}
        disabled={!sleeperUser.trim() || connStatus==="connecting"} onClick
        {connStatus==="connecting" ? "Looking up…" : "Connect Sleeper →"}
      </button>
    </div>
  </div>
)}

{step === 2 && plat === "yahoo" && (
  <div className="ob-card">
    <div className="ob-step"><span className="tag tag-green">Step 2 of 4 —
    <div className="ob-h">Connect Yahoo</div>
    <div className="ob-p">You'll be redirected to Yahoo to authorize access
    <div style={{background:"var(--bg)", border:"1px solid var(--border)",
      <div style={{fontSize:12, color:"var(--muted)", marginBottom:10}}>Wha
      <div style={{display:"flex", flexDirection:"column", gap:6}}>
        {["Read your fantasy leagues", "View your roster & standings", "No
          <div key={i} style={{fontSize:13, color:"var(--text)", display:"f
            <span style={{color:"var(--accent)"}}>✓</span>{f}
          </div>
        ))}
      </div>
```

```
    </div>
    <div className="ob-btns">
      <button className="btn btn-ghost" onClick={()=>setStep(1)}>← Back</bu
      <button className="btn btn-primary" style={{flex:1}}
        disabled={connStatus==="connecting"} onClick={connectYahoo}>
        {connStatus==="connecting" ? "Redirecting to Yahoo…" : "Authorize w
      </button>
    </div>
  </div>
)}

{step === 2 && plat === "espn" && (
  <div className="ob-card">
    <div className="ob-step"><span className="tag tag-green">Step 2 of 4 —
    <div className="ob-h">ESPN Cookies</div>
    <div className="ob-p">ESPN has no public OAuth API. To connect, paste y
    <div className="form-stack">
      <div className="form-field">
        <label className="form-label">ESPN_S2 Cookie</label>
        <input className="form-input" value={espnS2} onChange={e=>setEspnS2
      </div>
      <div className="form-field">
        <label className="form-label">SWID Cookie</label>
        <input className="form-input" value={espnSWID} onChange={e=>setEspn
      </div>
      <div className="form-field">
        <label className="form-label">League ID</label>
        <input className="form-input" value={espnLeagueId} onChange={e=>set
      </div>
      {connStatus==="error" && <div className="form-err">⚠ {connError}</div
    </div>
    <div style={{fontSize:11, color:"var(--muted)", marginBottom:20}}>🔒 Cc
    <div className="ob-btns">
      <button className="btn btn-ghost" onClick={()=>setStep(1)}>← Back</bu
      <button className="btn btn-primary" style={{flex:1}}
        disabled={connStatus==="connecting"} onClick={connectESPN}>
        {connStatus==="connecting" ? "Connecting…" : "Connect ESPN →"}
      </button>
    </div>
  </div>
)}

{step === 2 && plat === "nfl" && (
  <div className="ob-card">
    <div className="ob-step"><span className="tag tag-green">Step 2 of 4 —
    <div className="ob-h">NFL.com</div>
    <div className="ob-p">NFL.com fantasy support is coming soon. For now,
```

```
    <div style={{textAlign:"center", padding:"24px 0"}}>
      <div style={{fontSize:36, marginBottom:12}}>🚧</div>
      <div style={{fontSize:13, color:"var(--muted)"}}>Coming in v4</div>
    </div>
    <button className="btn btn-ghost" style={{width:"100%"}} onClick={()=>s
  </div>
)}

{/* ── STEP 3: League picker ── */}
{step === 3 && (
  <div className="ob-card">
    <div className="ob-step"><span className="tag tag-green">Step 3 of 4</s
    <div className="ob-h">Pick Your League</div>
    <div className="ob-p">We found {leagues.length} league{leagues.length!=
    <div style={{display:"flex", flexDirection:"column", gap:10, marginBott
      {leagues.map(l => (
        <div key={l.id}
          className={`plat-card${chosenLeague?.id===l.id?" active":""}`}
          style={{textAlign:"left", display:"flex", alignItems:"center", ju
          onClick={() => handleLeaguePick(l)}>
          <div>
            <div style={{fontSize:14, fontWeight:600, color:"var(--bright)"
            <div style={{fontSize:11, color:"var(--muted)"}}>{l.size} teams
          </div>
          <div style={{fontSize:20}}>{chosenLeague?.id===l.id ? "✓" : "→"}<
        </div>
      ))}
    </div>
    <button className="btn btn-ghost" style={{width:"100%"}} onClick={()=>s
  </div>
)}

{/* ── STEP 4: Scoring confirm ── */}
{step === 4 && chosenLeague && (
  <div className="ob-card">
    <div className="ob-step"><span className="tag tag-green">Step 4 of 4</s
    <div className="ob-h">Confirm Scoring</div>
    <div className="ob-p">We auto-detected your scoring format. Confirm or

    <div style={{background:"var(--bg)", border:"1px solid var(--border2)",
      <div style={{fontSize:11, color:"var(--muted)", marginBottom:4}}>Conn
      <div style={{fontSize:14, fontWeight:600, color:"var(--bright)"}}>{ch
      <div style={{fontSize:11, color:"var(--accent)", marginTop:3}}>
        {plat.charAt(0).toUpperCase()+plat.slice(1)} · {chosenLeague.size}
      </div>
    </div>
```

```
          <div className="form-stack">
            <div className="form-field">
              <label className="form-label">Scoring Format <span style={{color:"v
              <select className="form-select" value={scoring} onChange={e=>setSco
                <option>PPR</option>
                <option>Half PPR</option>
                <option>Standard</option>
                <option>2QB / Superflex</option>
                <option>TE Premium</option>
              </select>
            </div>
          </div>
          <div className="ob-btns">
            <button className="btn btn-ghost" onClick={()=>setStep(3)}>← Back</bu
            <button className="btn btn-primary" style={{flex:1}} onClick={handleF
              Launch Dashboard →
            </button>
          </div>
        </div>
      )}
    </div>
  );
}

// ── AGENT ORCHESTRATION PANEL ──
function AgentPanel({ scoring, onReady }) {
  const [agentState, setAgentState] = useState(
    Object.fromEntries(AGENT_DEFS.map(a => [a.id, "wait"]))
  );
  const [agentData, setAgentData] = useState({});
  const [progress, setProgress] = useState(0);
  const [verdict, setVerdict] = useState(null);
  const doneRef = useRef(false);

  useEffect(() => {
    const run = async () => {
      const ctx = {};
      for (let i = 0; i < AGENT_DEFS.length; i++) {
        const a = AGENT_DEFS[i];
        setAgentState(s => ({...s, [a.id]:"active"}));
        await new Promise(r => setTimeout(r, 900 + Math.random()*400));
        const result = a.done(scoring);
        ctx[a.id] = result;
        setAgentData(d => ({...d, [a.id]: result}));
        setAgentState(s => ({...s, [a.id]:"done"}));
        setProgress(i+1);
      }
```

```
    // Manager Agent synthesizes
    const move = await callManagerAgent({
      weather:   ctx.weather,
      travel:    ctx.travel,
      gametime:  ctx.gametime,
      roster:    ctx.roster,
      perf:      ctx.perf,
      matchup:   ctx.matchup,
      scoring,
      record:    "10-4"
    });
    setVerdict(move);
    setTimeout(() => onReady(move, ctx), 1800);
  };
  if (!doneRef.current) { doneRef.current = true; run(); }
}, []);

return (
  <div className="agent-panel">
    <div className="agent-scan" />
    <div className="agent-header">
      <div className="agent-title">
        <div className="pulse" />
        <span className="agent-ttl-txt">Agents Running — Week 15</span>
      </div>
      <span className="agent-prog-txt">{progress} / {AGENT_DEFS.length} complet
    </div>
    <div className="agent-steps">
      {AGENT_DEFS.map(a => {
        const st = agentState[a.id];
        return (
          <div key={a.id} className={`a-step s-${st}`}>
            <div className="a-step-ico">{a.ico}</div>
            <div className="a-step-body">
              <div className="a-step-name">
                {a.name}
                <span className="a-step-src">{a.src}</span>
              </div>
              <div className={`a-step-status s-${st}`}>
                {st==="done"   ? `✓ ${agentData[a.id] || a.done(scoring)}` :
                 st==="active" ? a.wait :
                 st==="error"  ? "✗ Failed — using fallback data" :
                 "Queued"}
              </div>
            </div>
          </div>
        );
```

```
        })}
      </div>
      {verdict && (
        <div className="verdict-box">
          <div className="verdict-lbl"><div className="pulse" style={{width:5,hei
          <div className="verdict-txt">{verdict.shortVerdict}</div>
          <div className="verdict-meta">
            <span>🎯 Confidence: <strong style={{color:"var(--accent)"}}>{verdict
            <span>📋 Type: {verdict.moveType}</span>
          </div>
        </div>
      )}
    </div>
  );
}

// ── ARCHITECTURE BANNER ──
function ArchBanner() {
  const [open, setOpen] = useState(null); // null | 'vault' | 'redis' | 'hitl'

  const panels = {
    vault: {
      title: "DB Security — Supabase Vault",
      color: "var(--accent)",
      content: [
        "access_token, refresh_token, espn_s2, espn_swid are encrypted at rest us
        "Enable once: CREATE EXTENSION IF NOT EXISTS pgsodium;",
        "Wrap writes: SELECT vault.create_secret(token, 'yahoo_token_' || user_id
        "Wrap reads:  SELECT decrypted_secret FROM vault.decrypted_secrets WHERE
        "If DB is breached, encrypted fields are unreadable without the root key
      ]
    },
    redis: {
      title: "Rate Limiting — Redis Cache (Upstash)",
      color: "var(--gold)",
      content: [
        "100 users in the same Sleeper league = 100 identical API calls without c
        "Key pattern: standings:{platform}:{leagueId} — TTL 5 min",
        "Key pattern: nfl_scores:{season}:{week}    — TTL 60 min (scores don't ch
        "Key pattern: player_info:nfl               — TTL 24 hr (roster changes o
        "Deploy: Upstash serverless Redis — free tier, Railway add-on, single env
        "Cache hit = <5ms response. Cache miss = live API call + populate. Saves
      ]
    },
    hitl: {
      title: "Human-in-the-Loop Calibration Gate",
      color: "var(--blue)",
```

```
    content: [
      "The Tuesday cron only scores moves where followed = true in the DB.",
      "If a user accepted the move card but never actually did it in their app,
      "FeedbackCard now requires 'Did you follow this move?' before stars can b
      "followed=false moves are logged for UX insight (why did they ignore it?)
      "Cron query: WHERE outcome='pending' AND followed=true AND created_at < l
      "Result: cleaner training data → better confidence calibration over time.
    ]
  }
};

const secCards = [
  { id:"vault", ico:"🔐", label:"DB Security",    sub:"Supabase Vault",  col:"v
  { id:"redis", ico:"⚡", label:"Rate Limiting",  sub:"Redis Cache",     col:"v
  { id:"hitl",  ico:"👤", label:"Human-in-Loop",  sub:"Calibration Gate",col:"v
];

return (
  <div className="arch-banner">
    <div className="arch-ttl">v3 Architecture — Production Hardened</div>

    {/* Flow diagram */}
    <div className="arch-flow" style={{marginBottom:16}}>
      {[
        {label:"Browser",         h:false},
        {label:"→",               arrow:true},
        {label:"/api/move",       h:true},
        {label:"→",               arrow:true},
        {label:"Redis Cache",     h:false},
        {label:"→",               arrow:true},
        {label:"Manager Agent",   h:true},
        {label:"→",               arrow:true},
        {label:"Vault DB",        h:false},
        {label:"→",               arrow:true},
        {label:"Tue Cron",        h:false},
      ].map((n,i) => n.arrow
        ? <div key={i} className="arch-arrow">→</div>
        : <div key={i} className={`arch-node${n.h?" highlight":""}`}>{n.label}<
      )}
    </div>

    {/* Three security optimization cards */}
    <div style={{display:"grid", gridTemplateColumns:"repeat(3,1fr)", gap:8}}>
      {secCards.map(c => (
        <div key={c.id}
          onClick={() => setOpen(open===c.id ? null : c.id)}
          style={{
```

```
              background: open===c.id ? `rgba(${c.col===`var(--accent)`?"0,214,14
              border: `1px solid ${open===c.id ? c.col : "var(--border)"}`,
              borderRadius:8, padding:"12px 14px", cursor:"pointer", transition:"
            }}>
            <div style={{fontSize:18, marginBottom:5}}>{c.ico}</div>
            <div style={{fontSize:11, fontWeight:600, color:"var(--bright)", marg
            <div style={{fontSize:10, color:"var(--muted)"}}>{c.sub}</div>
            <div style={{fontSize:10, color: c.col, marginTop:6}}>{open===c.id?"▲
          </div>
        ))}
      </div>

      {/* Expanded detail panel */}
      {open && panels[open] && (
        <div style={{
          marginTop:10, padding:"16px 18px",
          background:"var(--bg)", border:`1px solid ${panels[open].color}`,
          borderRadius:8, animation:"fadeUp .2s ease"
        }}>
          <div style={{fontSize:11, fontWeight:700, letterSpacing:"2px", textTran
            color: panels[open].color, marginBottom:12}}>
            {panels[open].title}
          </div>
          <div style={{display:"flex", flexDirection:"column", gap:7}}>
            {panels[open].content.map((line, i) => (
              <div key={i} style={{display:"flex", gap:10, alignItems:"flex-start
                <span style={{color: panels[open].color, flexShrink:0, fontSize:1
                <span style={{
                  fontSize:11, color:"var(--text)", lineHeight:1.55,
                  fontFamily: line.startsWith("CREATE") || line.startsWith("SELEC
                  color: line.startsWith("CREATE") || line.startsWith("SELECT") |
                }}>{line}</span>
              </div>
            ))}
          </div>
        </div>
      )}
    </div>
  );
}

// ── MOVE CARD ──
function MoveCard({ move, scoring, onAccept, onSkip }) {
  const signals = [
    { ico:"🌦️", lbl:"Weather",  val:"Clear / 52°F",   sub:"Ideal run conditions"
    { ico:"⚔️",  lbl:"Matchup",  val:"#31 vs RB",      sub:"Bengals worst run D"
    { ico:"📈",  lbl:"Trend",    val:"+9.1 pts/wk",    sub:"3-wk above projection
```

```
  ];
  return (
    <div className="move-card">
      <div className="mc-head">
        <div className="mc-left">
          <div className="mc-eyebrow">
            <span className="tag tag-green">◆ {move.moveType}</span>
            <span className="tag tag-gold">Week 15</span>
          </div>
          <div className="mc-h">{move.headline}</div>
          <div className="mc-scoring">Optimized for {scoring} scoring</div>
        </div>
        <div className="mc-conf">
          <div className="mc-conf-lbl">Confidence</div>
          <div className="mc-conf-num">{move.confidence}</div>
          <div className="mc-conf-of">/ 100</div>
        </div>
      </div>
      <div className="mc-body">
        <div className="mc-reasoning">{move.reasoning}</div>
        <div className="signals">
          {signals.map((s,i) => (
            <div key={i} className="sig">
              <div className="sig-ico">{s.ico}</div>
              <div className="sig-lbl">{s.lbl}</div>
              <div className="sig-val">{s.val}</div>
              <div className="sig-sub">{s.sub}</div>
            </div>
          ))}
        </div>
        <div className="mc-actions">
          <button className="btn btn-primary" onClick={onAccept}>✓ Accept This Mo
          <button className="btn btn-danger" onClick={onSkip}>Skip This Week</but
          <span className="mc-note">1 move remaining this week</span>
        </div>
      </div>
    </div>
  );
}

// ── FEEDBACK CARD — Human in the Loop (v3) ──
// The critical insight: if a user accepts but doesn't actually execute
// the move (didn't make the waiver claim, forgot to start the player),
// scoring the outcome against them pollutes the calibration data.
// This toggle gates the cron — only followed=true moves get eff-scored.
function FeedbackCard({ type, move, onSubmit }) {
  const [stars,    setStars]    = useState(0);
```

```
const [note,     setNote]     = useState("");
const [followed, setFollowed] = useState(null); // null | true | false

const isAccept = type === "accept";
const ico      = isAccept ? "✅" : "📋";
const title    = isAccept ? "Move Accepted & Logged" : "Move Skipped — Noted";
const desc     = isAccept
  ? "Before we start tracking — did you actually make this move in your league
  : "Got it. Skipped moves still help us understand confidence calibration.";

const canSubmit = isAccept ? (followed !== null && stars > 0) : stars > 0;

return (
  <div className="state-card">
    <div className="state-ico">{ico}</div>
    <div className="state-h">{title}</div>
    <div className="state-p">{desc}</div>

    {/* ── HUMAN-IN-THE-LOOP TOGGLE (accept path only) ── */}
    {isAccept && (
      <div style={{
        background:"var(--bg)", border:`1px solid ${followed===null?"var(--bord
        borderRadius:10, padding:"20px 20px 16px", marginBottom:16,
        transition:"border-color .3s"
      }}>
        <div style={{fontSize:11, letterSpacing:"2px", textTransform:"uppercase
          ◆ Did You Actually Follow This Move?
        </div>
        <div style={{display:"flex", gap:10, marginBottom:12}}>
          <button
            onClick={() => setFollowed(true)}
            style={{
              flex:1, padding:"12px 0", borderRadius:7, border:"1px solid",
              borderColor: followed===true ? "var(--accent)" : "var(--border)",
              background:  followed===true ? "rgba(0,214,143,.1)" : "var(--surf
              color:       followed===true ? "var(--accent)" : "var(--muted)",
              fontFamily:"var(--font-b)", fontWeight:600, fontSize:13,
              cursor:"pointer", transition:"all .2s"
            }}>
            ✓ Yes — I made the move
          </button>
          <button
            onClick={() => setFollowed(false)}
            style={{
              flex:1, padding:"12px 0", borderRadius:7, border:"1px solid",
              borderColor: followed===false ? "var(--red)" : "var(--border)",
              background:  followed===false ? "rgba(255,77,106,.08)" : "var(--s
```

```
            color:       followed===false ? "var(--red)" : "var(--muted)",
            fontFamily:"var(--font-b)", fontWeight:600, fontSize:13,
            cursor:"pointer", transition:"all .2s"
          }}>
          ✗ No — I skipped it
        </button>
      </div>
      {followed === true && (
        <div style={{fontSize:11, color:"var(--accent)", display:"flex", alig
          ✓ Great — Tuesday's cron will score this against real NFL results.
        </div>
      )}
      {followed === false && (
        <div style={{fontSize:11, color:"var(--muted)", display:"flex", align
          Noted. This move will be logged but <strong style={{color:"var(--go
        </div>
      )}
    </div>
  )}

  {/* ── STAR RATING + NOTES ── */}
  <div className="feedback-prompt">
    <div className="fb-lbl">Rate This Recommendation</div>
    <div className="fb-stars">
      {[1,2,3,4,5].map(n => (
        <span key={n} className={`fb-star${stars>=n?" lit":""}`} onClick={()
      ))}
    </div>
    {stars > 0 && (
      <div style={{fontSize:11, color:"var(--muted)", marginBottom:10, margin
        {["","Poor — way off","Below average","Reasonable","Good call","Perfe
      </div>
    )}
    <textarea
      className="fb-inp"
      placeholder="Optional: context the AI should know? Injuries heard, news
      value={note}
      onChange={e => setNote(e.target.value)}
    />
    <button
      className="btn btn-primary fb-submit"
      onClick={() => onSubmit(stars, note, followed)}
      disabled={!canSubmit}>
      Submit & Lock In →
    </button>
    {isAccept && followed === null && (
      <div style={{fontSize:11, color:"var(--muted)", marginTop:8, textAlign:
```

```
            ↑ Tell us if you followed the move to enable accuracy tracking
          </div>
        )}
      </div>
    </div>
  );
}

// ── HISTORY ──
function HistPanel({ history, loading, error }) {
  const wins   = history.filter(h => h.outcome==="win").length;
  const losses = history.filter(h => h.outcome==="loss").length;
  const scored = history.filter(h => h.eff != null);
  const avgEff = scored.length ? scored.reduce((a,h) => a + h.eff, 0) / scored.le

  return (
    <div className="hist-panel">
      <div className="hist-head">
        <span className="hist-ttl">Move History + Effectiveness</span>
        <span className="hist-stat">
          {loading ? "Loading…" : error ? "⚠ Error" : `${wins}W-${losses}L · ${Ma
        </span>
      </div>
      {loading && (
        <div style={{padding:"28px", textAlign:"center", color:"var(--muted)", fo
          <div style={{marginBottom:8}}>⟳ Fetching from database…</div>
          <div style={{fontSize:11, opacity:.6}}>GET /api/moves → Supabase</div>
        </div>
      )}
      {error && (
        <div style={{padding:"20px", textAlign:"center"}}>
          <span className="tag tag-red">⚠ {error}</span>
          <div style={{fontSize:12, color:"var(--muted)", marginTop:8}}>Showing c
        </div>
      )}
      {!loading && (
        <div className="hist-list">
          {history.map((h,i) => {
            const col = h.eff >= 70 ? "var(--accent)" : h.eff >= 40 ? "var(--gold
            return (
              <div key={h.id||i} className="hist-item">
                <div className="hi-top">
                  <span className="hi-wk">
                    {h.week} · {h.type||"Move"}
                    {h.source==="db" && <span style={{marginLeft:6, fontSize:9, c
                    {h.followed===true  && <span style={{marginLeft:5, fontSize:9
                    {h.followed===false && <span style={{marginLeft:5, fontSize:9
```

```
                  </span>
                  <span className="hi-eff" style={{color: h.eff ? col : "var(--mu
                    {h.eff ? `${h.eff}%` : "Pending"}
                  </span>
                </div>
                <div className="hi-mv">{h.move}</div>
                <div className="hi-out">
                  {h.outcome==="win"     && <span className="hi-win">✓ Won</span>
                  {h.outcome==="loss"    && <span className="hi-loss">✗ Lost</spa
                  {h.outcome==="pending" && <span className="hi-pend">⏳ Pending
                  <span style={{fontSize:11}}>— {h.result}</span>
                </div>
                {h.eff != null && (
                  <div className="eff-track">
                    <div className="eff-fill" style={{width:`${h.eff}%`, backgrou
                  </div>
                )}
              </div>
            );
          })}
        </div>
      )}
    </div>
  );
}

// ── STANDINGS ──
function StandPanel({ platform, leagueId, teamName }) {
  const [standings, loading, error] = useStandings(platform, leagueId, teamName);

  return (
    <div className="stand-panel">
      <div className="stand-head">
        <div style={{display:"flex", alignItems:"center", justifyContent:"space-b
          <div className="stand-lbl">League Standings</div>
          {loading && <span style={{fontSize:11, color:"var(--muted)"}}>⟳ Live</s
          {error   && <span className="tag tag-red" style={{fontSize:9}}>⚠ Cached
          {!loading && !error && <span style={{fontSize:10, color:"var(--accent)"
            {platform?.toUpperCase()} LIVE
          </span>}
        </div>
      </div>
      {loading && (
        <div style={{padding:"20px", textAlign:"center", color:"var(--muted)", fo
          Fetching from {platform} API…
        </div>
      )}
```

```
      {!loading && standings.map((r,i) => (
        <div key={i} className={`stand-row${r.me?" me":""}`}>
          <span className="s-rank">{r.rank}</span>
          <span className="s-name">{r.name}</span>
          <span className="s-rec">{r.rec}</span>
          <span className="s-pts">{r.pts}</span>
        </div>
      ))}
    </div>
  );
}

// ── SUBSCRIPTION MODAL ──
function SubModal({ onClose, onSubscribe }) {
  return (
    <div className="overlay" onClick={e => e.target===e.currentTarget && onClose(
      <div className="modal">
        <button className="modal-x" onClick={onClose}>×</button>
        <div className="modal-eyebrow"><span className="tag tag-gold">🍺 Slops Sa
        <div className="modal-h">Go MVP</div>
        <div className="modal-p">One AI-crafted move every week from the Saloon's
        <div className="plan-grid">
          <div className="plan">
            <div className="plan-name">Monthly</div>
            <div className="plan-price">$9</div>
            <div className="plan-per">per month</div>
            <ul className="plan-feats">
              <li className="plan-feat">1 AI move / week</li>
              <li className="plan-feat">All 6 agents</li>
              <li className="plan-feat">Move history</li>
              <li className="plan-feat">Effectiveness tracking</li>
            </ul>
          </div>
          <div className="plan best">
            <div className="plan-badge">Best Value</div>
            <div className="plan-name">Season Pass</div>
            <div className="plan-price">$49</div>
            <div className="plan-per">full season (~18 wks)</div>
            <ul className="plan-feats">
              <li className="plan-feat">1 AI move / week</li>
              <li className="plan-feat">All 6 agents + Manager</li>
              <li className="plan-feat">Full history + analytics</li>
              <li className="plan-feat">Tuesday cron outcomes</li>
              <li className="plan-feat">Scoring format memory</li>
            </ul>
          </div>
        </div>
```

```
        <button className="btn btn-primary" style={{width:"100%"}} onClick={() =>
          Start Free — 7 Day Trial (Stripe)
        </button>
        <div className="modal-fine">Powered by Stripe. Secure. Cancel anytime.</d
      </div>
    </div>
  );
}

// ── SUBSCRIPTION GATE ──
function SubGate({ onSub }) {
  return (
    <div className="gate">
      <div className="gate-lock">🍺</div>
      <div className="gate-h">Saloon Members Only</div>
      <div className="gate-p">Subscribe to unlock your weekly AI move from the Sl
      <div className="gate-features">
        <div className="gate-feat">6 AI Agents + Manager</div>
        <div className="gate-feat">Self-Improving Model</div>
        <div className="gate-feat">🔐 Vault Encrypted</div>
        <div className="gate-feat">⚡ Redis Cached</div>
      </div>
      <button className="btn btn-primary" onClick={onSub}>Join the Saloon — $9/mo
    </div>
  );
}

// ── DASHBOARD ──
function Dashboard({ user, onSub }) {
  // For demo: subscriber toggle
  const [isSubscriber, setIsSubscriber] = useState(true);
  const [phase, setPhase] = useState("agents"); // agents | move | feedback | don
  const [move, setMove] = useState(null);
  const [fbDone, setFbDone] = useState(false);
  const [toast, showToast] = useToast();

  // Real data hooks — swap fetchHistory/fetchStandings internals for production
  const [history, histLoading, histError, addMove, updateOutcome] =
    usePersistentHistory(user.leagueId, user.platform);

  const handleAgentsReady = (m, _ctx) => {
    setMove(m);
    setPhase("move");
  };

  const handleAccept = () => {
    setPhase("feedback-accept");
```

```
  const newMove = {
    id: Date.now(), week:"Week 15", wkNum:15,
    move: move.headline, type: move.moveType,
    outcome:"pending", result:"Tracking — check back Tuesday", eff: null,
    scoring: user.scoring, source:"live",
    followed: null  // set after user confirms in FeedbackCard
  };
  addMove(newMove);
  showToast("Move logged! Tell us if you followed it to enable tracking.");
};

const handleSkip = () => {
  setPhase("feedback-skip");
  const newMove = {
    id: Date.now(), week:"Week 15", wkNum:15,
    move: `[Skipped] ${move.headline}`, type: move.moveType,
    outcome:"pending", result:"Skipped — outcome still tracked", eff: null,
    scoring: user.scoring, source:"live",
    followed: false
  };
  addMove(newMove);
  showToast("Skipped and logged for calibration.");
};

const handleFbSubmit = (stars, note, followed) => {
  // Update the most recent move's followed flag in history
  // PRODUCTION: PATCH /api/moves/:id { user_stars: stars, user_note: note, fol
  setFbDone(true);
  const followedMsg = followed === true
    ? "Tracking enabled — we'll score this Tuesday. 🎯"
    : followed === false
    ? "Noted — excluded from calibration to keep data clean."
    : "Feedback saved!";
  showToast(followedMsg);
  setTimeout(() => setPhase("done"), 800);
};

return (
  <div className="dash">
    <div className="topbar">
      <div className="tb-logo">
        SSFFMVP
        <span style={{fontSize:10, fontFamily:"var(--font-b)", fontWeight:500,
          letterSpacing:"2px", textTransform:"uppercase", marginLeft:8,
          background:"rgba(245,197,66,.1)", border:"1px solid rgba(245,197,66,.
          padding:"2px 8px", borderRadius:3}}>
          🍺 Slops Saloon
```

```
      </span>
    </div>
    <div className="tb-right">
      <span className="tag tag-green tb-week">Week 15</span>
      <span className="tb-team">{user.teamName}</span>
      <span className="tb-rec">10-4</span>
      <button className={`sub-pill${isSubscriber?"":" locked"}`} onClick={()
        {isSubscriber ? "✓ Pro" : "Upgrade →"}
      </button>
    </div>
  </div>
  <div className="dash-grid">
    {/* LEFT COLUMN */}
    <div>
      <div className="sec-head"><div className="sec-lbl">System Architecture<
      <ArchBanner />

      <div className="sec-head"><div className="sec-lbl">This Week's Move</di

      {!isSubscriber && <SubGate onSub={onSub} />}

      {isSubscriber && phase === "agents" && (
        <AgentPanel scoring={user.scoring} onReady={handleAgentsReady} />
      )}
      {isSubscriber && phase === "move" && move && (
        <MoveCard move={move} scoring={user.scoring} onAccept={handleAccept}
      )}
      {isSubscriber && phase === "feedback-accept" && !fbDone && (
        <FeedbackCard type="accept" move={move} onSubmit={handleFbSubmit} />
      )}
      {isSubscriber && phase === "feedback-skip" && !fbDone && (
        <FeedbackCard type="skip" move={move} onSubmit={handleFbSubmit} />
      )}
      {isSubscriber && phase === "done" && (
        <div className="state-card">
          <div className="state-ico">🏆</div>
          <div className="state-h">All Set for Week 15</div>
          <div className="state-p">Your move is logged. Tuesday's cron job wi
        </div>
      )}

      <div className="sec-head" style={{marginTop:22}}><div className="sec-lb
      <StandPanel platform={user.platform} leagueId={user.leagueId} teamName=
    </div>

    {/* RIGHT COLUMN */}
    <div>
```

```
          <div className="sec-head"><div className="sec-lbl">Move History + Feedb
          <HistPanel history={history} loading={histLoading} error={histError} />
        </div>
      </div>
      <Toast toast={toast} />
    </div>
  );
}

/* ═══════════════════════════════════════
   ROOT APP
═══════════════════════════════════════ */
export default function SlopsSaloonFFMVP() {
  const [screen, setScreen] = useState("landing");
  const [user, setUser]     = useState(null);
  const [showSub, setShowSub] = useState(false);
  const [toast, showToast]  = useToast();

  const handleSubscribe = (plan) => {
    // PRODUCTION: POST /api/stripe/checkout-session → redirect to Stripe
    showToast(`Redirecting to Stripe for ${plan} plan…`);
    setTimeout(() => setShowSub(false), 1500);
  };

  return (
    <>
      <style>{CSS}</style>
      <div className="app">
        {screen==="landing"  && <Landing onStart={()=>setScreen("ob")} onSub={()=
        {screen==="ob"       && <Onboarding onDone={u=>{setUser(u);setScreen("das
        {screen==="dash" && user && (
          <Dashboard user={user} onSub={()=>setShowSub(true)} />
        )}
        {showSub && <SubModal onClose={()=>setShowSub(false)} onSubscribe={handle
        <Toast toast={toast} />
      </div>
    </>
  );
}
```